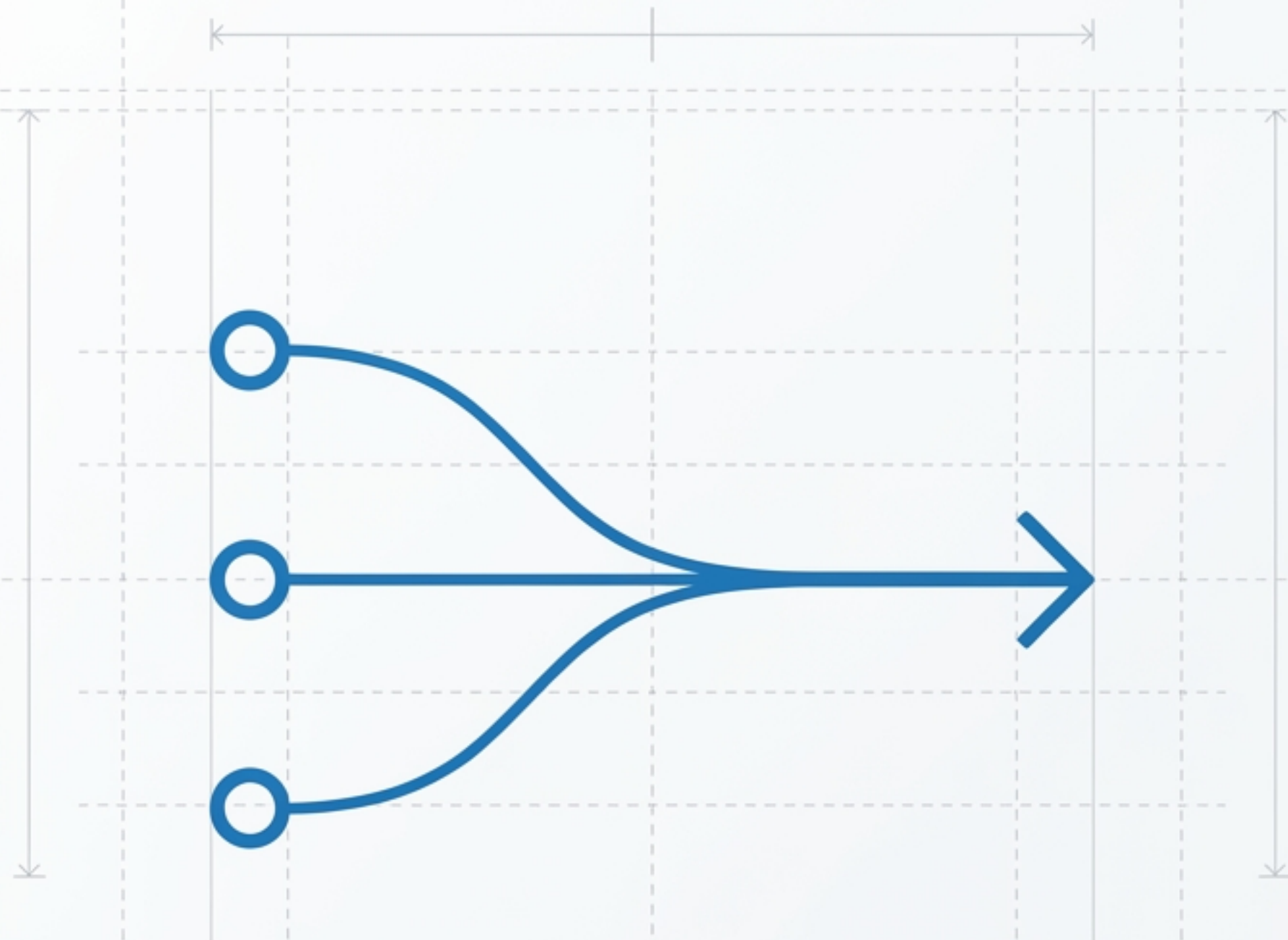




Unified Service Client Architecture (v0.33.1)

Establishing Stateless Facades and Unified Transport for FastAPI Clients

Status: Proposed | Jan 2025

Affected: `osbot_fast_api`, `mgraph_ai_service_cache`, `mgraph_ai_service_html_graph`

Executive Summary: From Instance Store to Config Store

The Key Insight

The Registry now stores configs keyed by client type, rather than storing client instances.

The Technical Shift

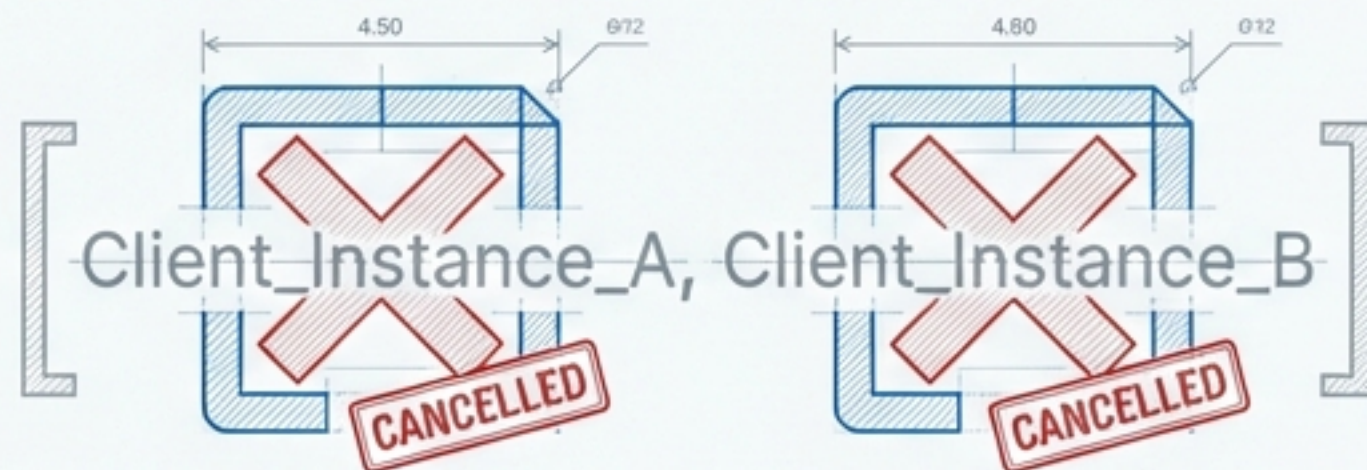
Client classes transform into stateless facades. They look up their configuration from the registry only at request time.

The Win

- ⊕ Seamless switching between IN_MEMORY and REMOTE modes.
- ⊕ Zero code changes required in the client implementation.
- ⊕ Elimination of redundant wrapper classes.

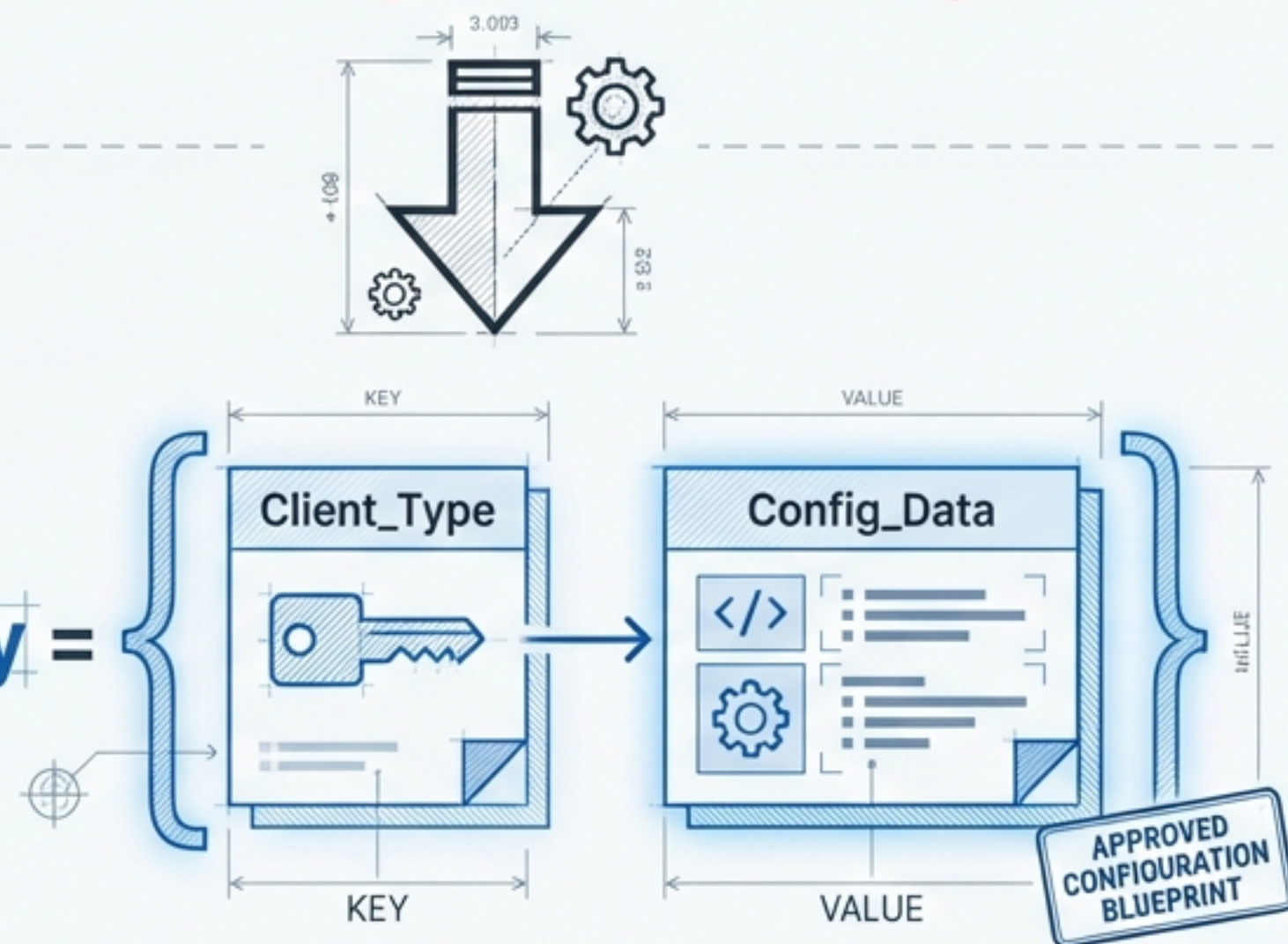
Before:

Registry =

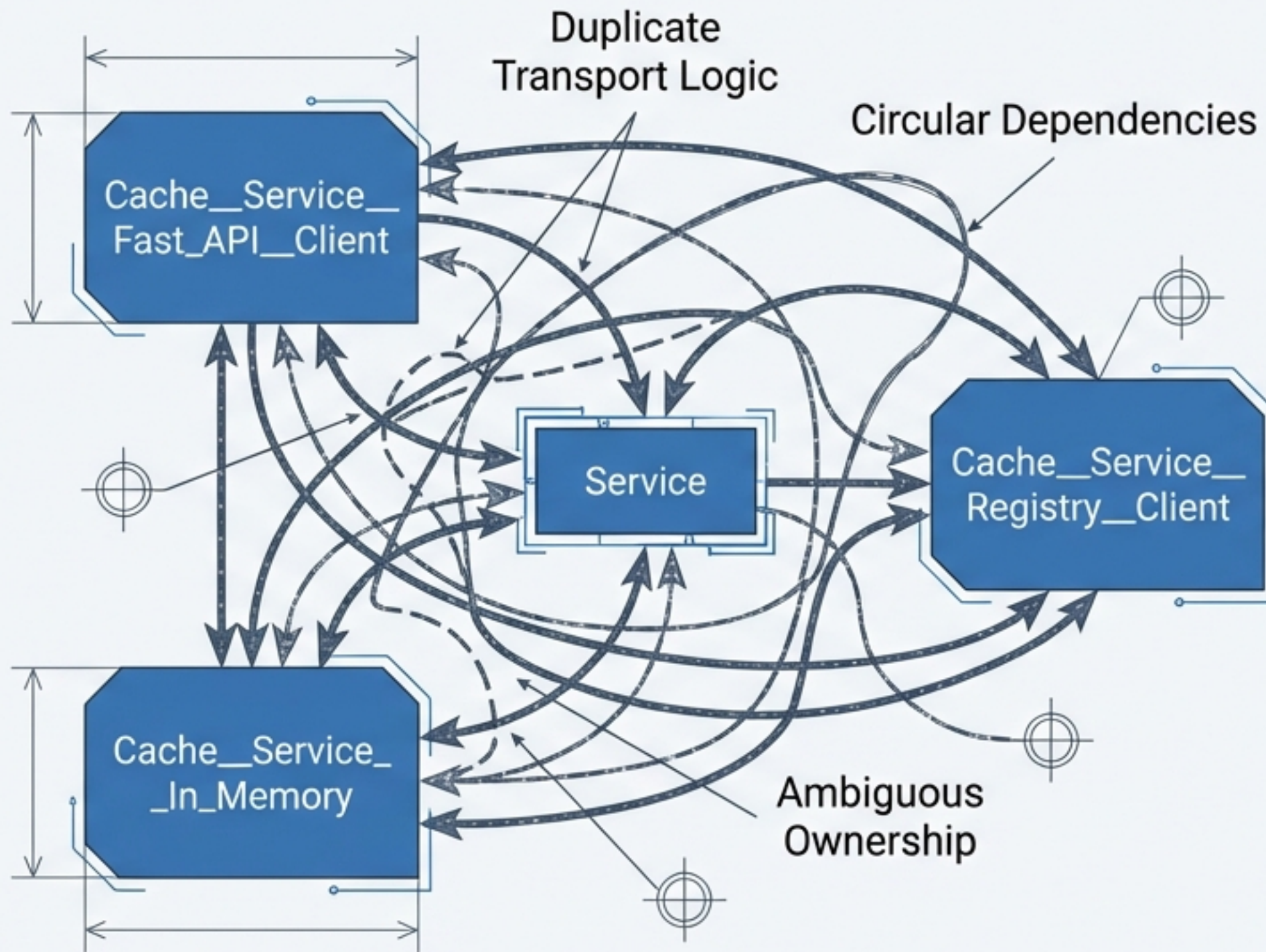


After:

Registry =



The Current State: Organic Complexity



Redundancy

3+ classes performing the job of one.

Inconsistency

Implementation patterns vary significantly across services.

Duplication

Configuration fields repeated between registry and client objects.

Blurry Boundaries

Unclear ownership of the FastAPI app in IN_MEMORY mode.

Investigation Phase: The Wrapper Problem

The Hard Question:

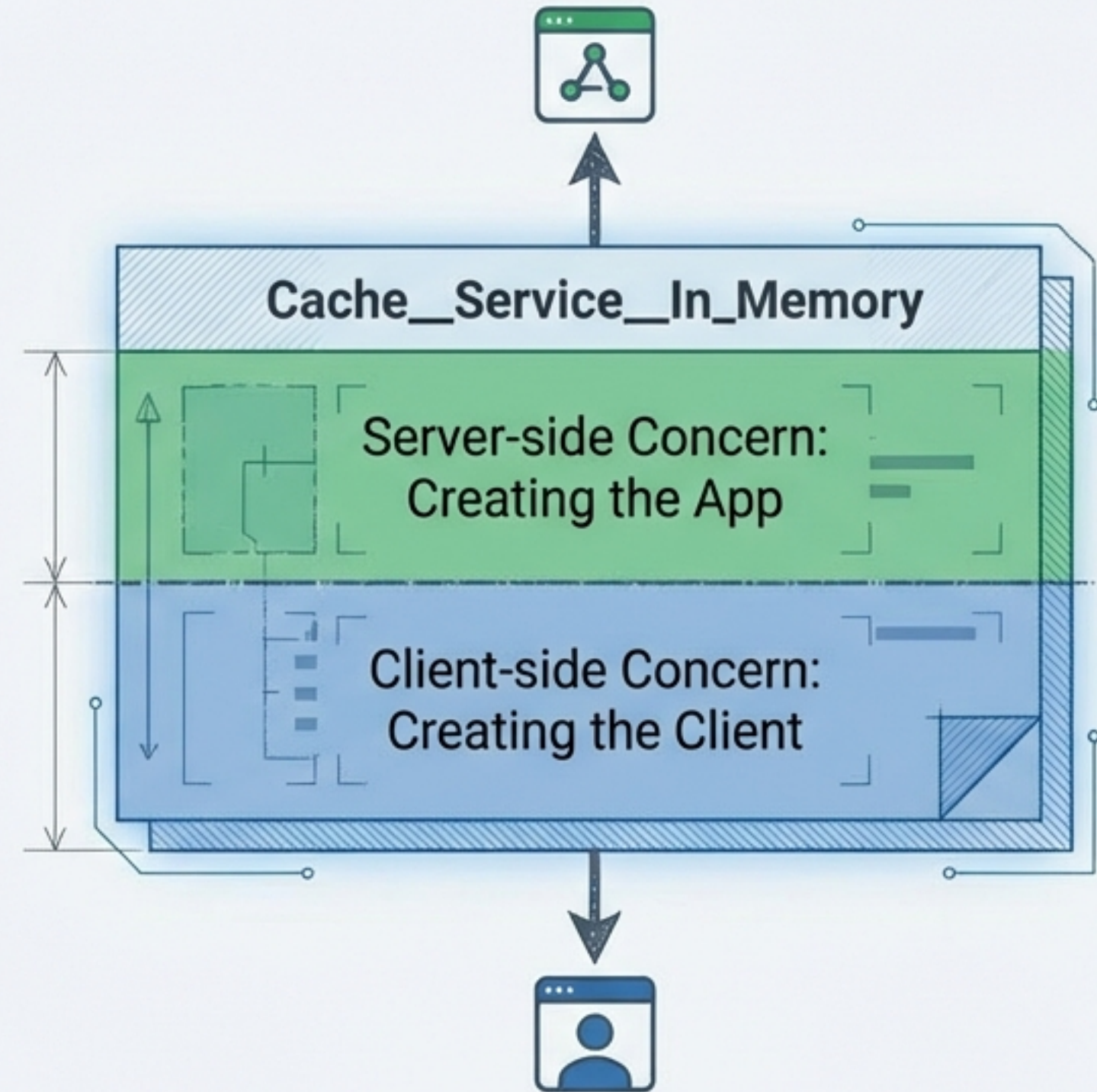
Why do we need the class
"Cache__Service__In_Memory"?

The Analysis - Illustrated as a dissected code block

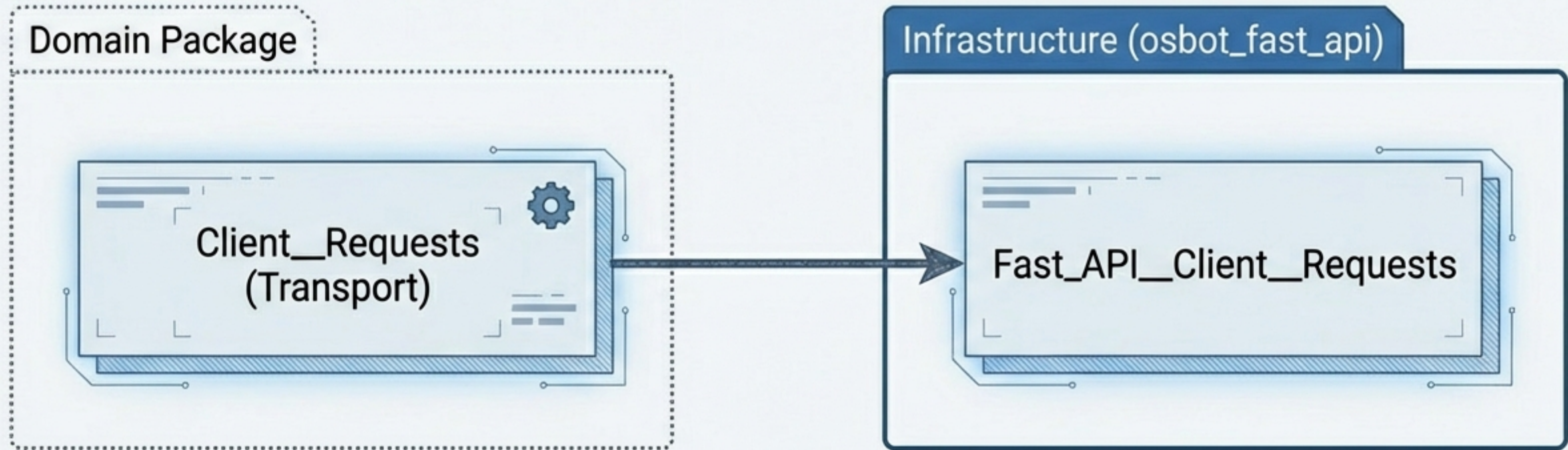
The Conclusion:

Verdict: This class is bootstrapping glue.

Decision: Replace wrapper classes with a simple registration function.



Discovery: Transport Logic is Generic

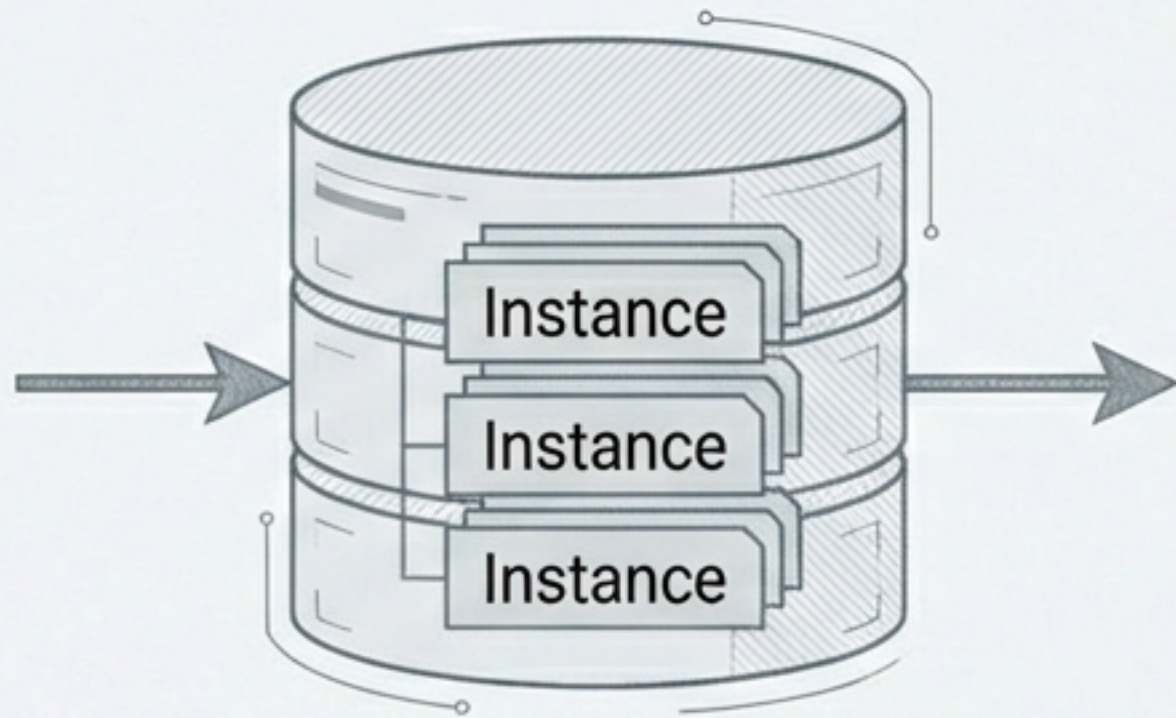


Insight: Audit of "Cache_Service_Fast_API_Client_Requests" revealed zero domain-specific logic.

Action: Promote logic to infrastructure layer. Share single transport class across all clients.

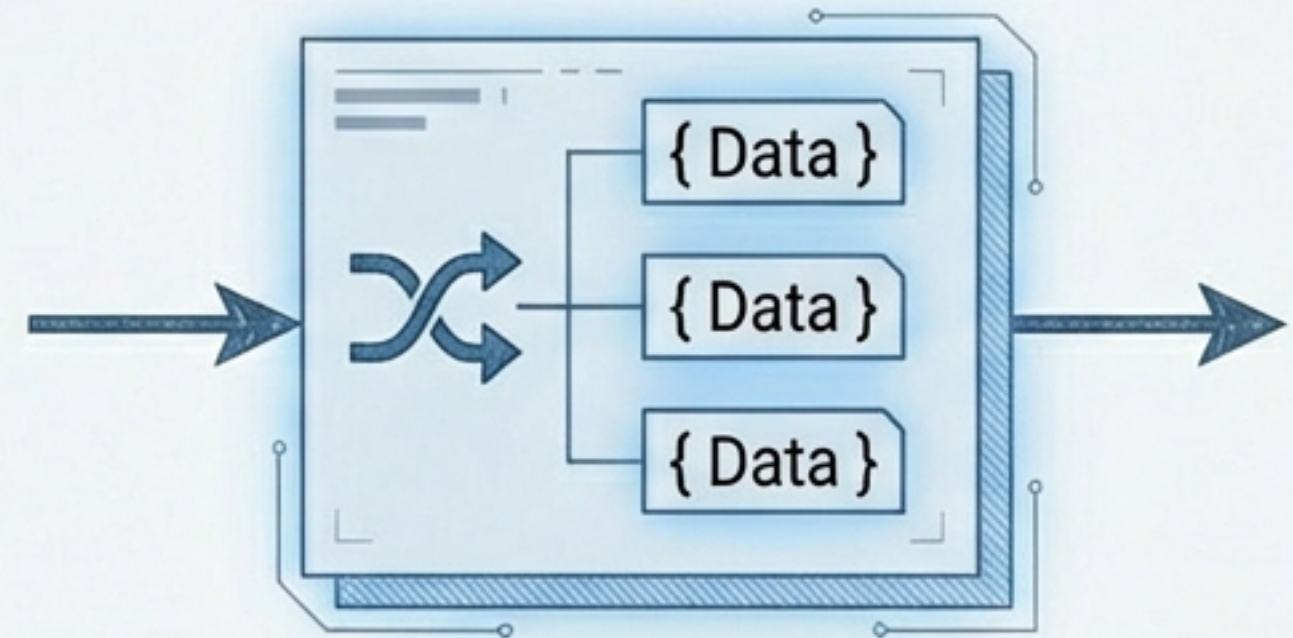
The Pivot: A New Mental Model

Old Model: Registry holds Clients



- Leads to statefulness.
- Rigid instantiation.
- Circular dependencies.

New Model: Registry holds Configs



- **Stateless Facades:** Clients fetch config once per request.
- **Decoupling:** Domain operations receive a "requests" object, not a client instance.
- **Clarity:** Answers "What config does this type use?"

The Final Architecture: Layered & Decoupled

Domain Client

Cache_Service_Client (Stateless Facade)

Generic Transport

Fast_API_Client_Requests (Shared Logic)

Registry

Fast_API_Service_Registry (Config Store)

Service Package 
(mgraph)
Handles Registration
Helpers (Wiring)

Component Responsibilities & Ownership

osbot_fast_api

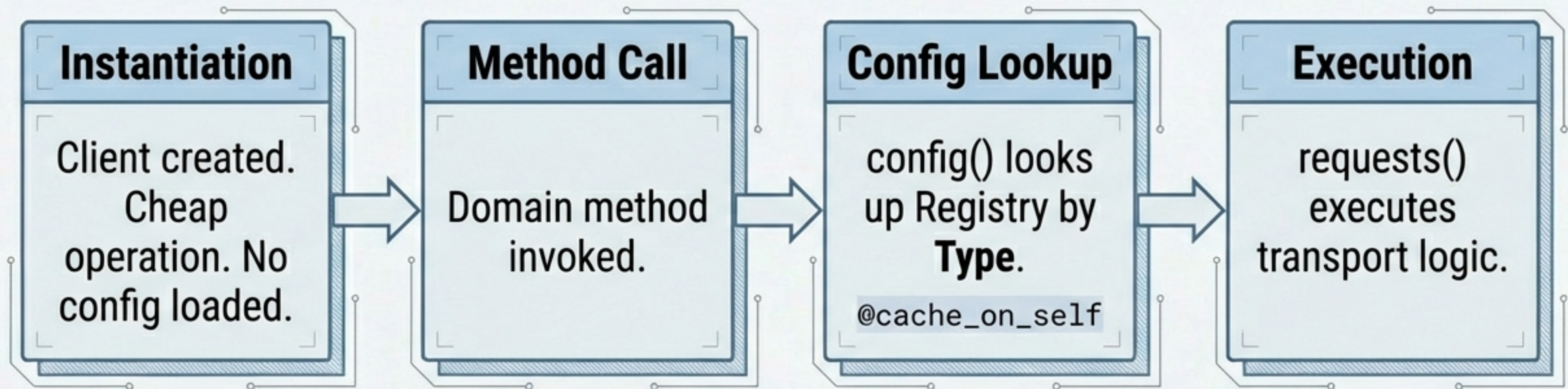
- Owns transport logic ('Fast_API_Client_Requests').
- Owns the Registry and Base Classes.
- Defines the Configuration Schema.

mgraph_ai_service_client

- Contains Stateless Facades.
- Owns Domain Methods only.
- Never imports from the Service package (One-way dependency).

mgraph_ai_service owns the wiring/registration helpers for In-Memory mode.

Pattern Deep Dive: The Stateless Facade



Technical Note: The '`config()`' method uses '`@cache_on_self`'. This ensures the registry lookup happens only once per instance, preventing performance hits on repeated access.

Unified Usage: One Codebase, Two Modes

REMOTE Mode (Production)

- Standard HTTP Transport
- Installed: **Client Package Only**
- Config: URL + Headers

IN_MEMORY Mode (Testing)

- Direct call to FastAPI (TestClient)
- Installed: **Client + Service Packages**
- Config: 'fast_api_app' Object
- Config: 'fast_api_app' Object

The Client Code Remains Identical.

The difference is solely in the Registry Configuration.

Migration Strategy

01. Infrastructure Update

Target: 'osbot_fast_api'

- Update Registry to store configs.
- Add 'Dict_Fast_API_Service_Configs_By_Type'.
- Add 'Fast_API_Client_Requests'.

02. Domain Client Update

Target: 'mgraph_ai_service_client'

- Create 'Cache_Service_Client'.
- Update domain operations: Change '_client' to 'requests'.

03. Service Wiring

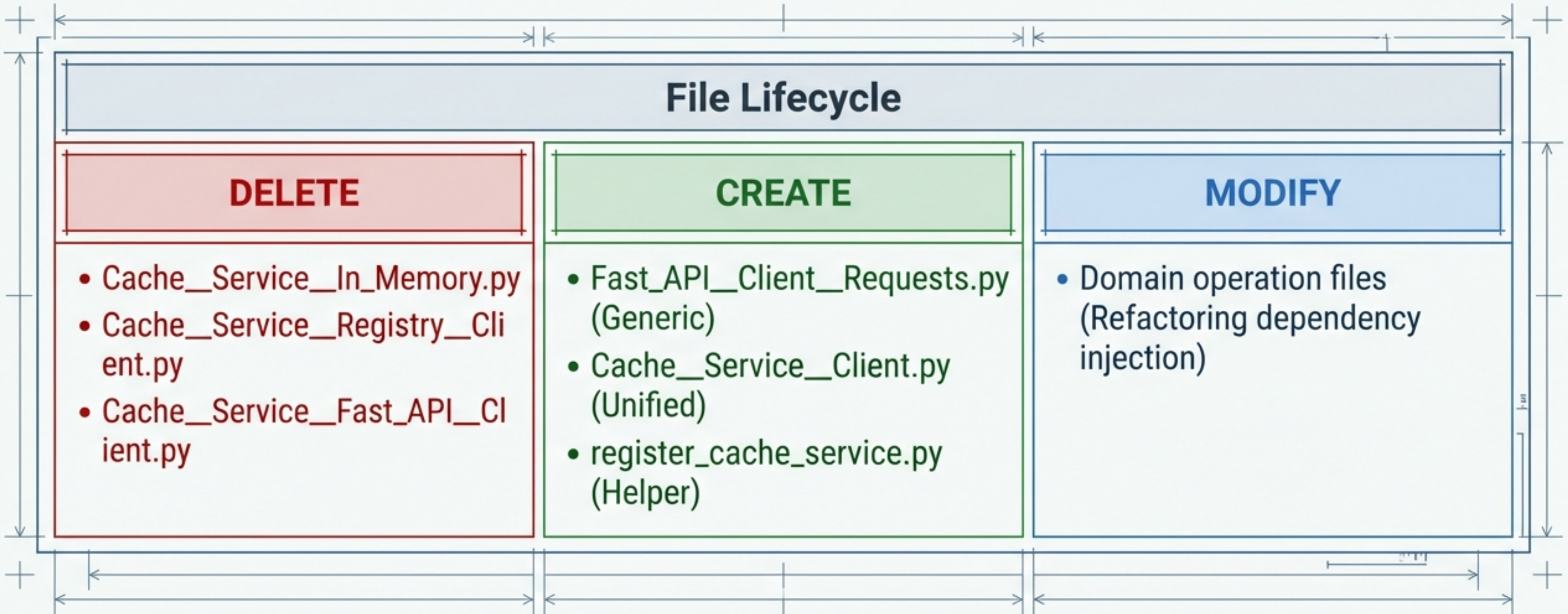
Target: 'mgraph_ai_service'

- Create 'registration/register_cache_service.py'.

04. Cleanup

- Delete 'In_Memory' and old wrapper files.
- Update tests.

The Cleanup: Reducing the Footprint



Benefits of Unification

1. Reduced Complexity

- 1 class per service client (down from 3+).
- Shared transport logic via osbot infrastructure.

2. Performance

- ~100ms startup time for full service stack.
- Efficient config caching.

3. Testability

- No mocks required.
- IN_MEMORY mode uses real FastAPI app.
- Tests run on production code paths.

4. Clear Ownership

- Sharp boundaries between Infrastructure (osbot) and Domain (mgraph).

Open Questions & Future Considerations

1. Global vs. Instance Registry

- Current implementation is a global singleton.
- Should we support per-application registries for stricter isolation?

2. Error Handling

- Convention decision: Should the transport layer raise exceptions or return structured error results?

3. Async Support

- Requirement check: Do we need to add async capabilities to 'Fast_API_Client_Requests'?

4. Config Validation

- Strictness: Should the registry validate config completeness upon registration (e.g. enforcing 'fast_api_app' presence)?

Ready for Scale

“By making clients cheap, stateless, and uniform, we prepare the architecture for rapid service expansion.”

Current Status:

- **Current Status:** Infrastructure changes ready for review.

Next Step:

- **Next Step:** Begin Phase 1 migration of 'mgraph_ai_service_cache'.